Generic mixed-degree addition: literal word correspondence

Five theorems compiled for the retained generic degree-one plus degree-two branch. The proof preserves the source multiplication nesting, including the different outer-dispatch and generic-body expressions for k. No exponentiation loop occurs in this branch: every square is an explicit multiplication.

1. generic_word_decode: all six output words and k are canonical residues; decoded values equal MixedGeneric.output and genericK.
2. generic_guard_decode: the actual word k is zero exactly when the decoded field k is zero.
3. generic_dispatch_decode: mapping the word Option result through decoding gives the field Option dispatch, covering both None and Some.
4. generic_outer_guard: the outer complete-dispatch k word equals the generic-body k word, despite their different multiplication associations.
5. generic_word_preserves: with valid normalized inputs, four nonzero weights and nonzero k, dispatch returns a canonical degree-two payload with nonzero output weights and normalized Mumford divisibility.

Value assumptions: UInt64 modulus p, prime with 5 <= p < 2^31; all ten coordinate words and curve[3] are canonical. The literal output w=1 is included in canonicality and decoding. The validity theorem takes the remaining three curve coefficients directly as field values; they are not read by this machine branch. Its curve is the monic quintic with no X^4 term. Input validity and nonzero weights are premises, not output-validity assumptions.

Source binding: projective_law_gate.rs lines 232–292 for generic addition and 828–851 for outer dispatch; Field arithmetic is the retained n=1, tables=None path with sufficient normal counter headroom. FieldExpr establishes bounded arithmetic value correspondence. These theorems do not establish compiler behavior, Rust execution traces, operation/counter equivalence, divisor-class addition, or a whole collector walk. The pure genericWords definition also defines unused payload values when k=0; Option correspondence does not assert that those computations run in the early-return Rust branch.

Evidence: main attempt3, Controls controls4 and separate Readback readback1 all exited zero under the pinned Lean toolchain. Five axiom readbacks contain only propext, Classical.choice and Quot.sound. Attempt1 failed on literal-one normalization; controls1–3 retain their canonical-predicate and finite-field normalization failures. All failures remain in this packet.

Controls: a nonunit-weight F7 fixture invokes the full validity theorem, using a=(0,1,2,2), b=(0,4,0,0,3,2), and f=X^5+X^2-X+1. Separate controls check canonical payload, nonzero generic k, k=0 returning None, rejected canonicality at y=p, and zero input z yielding k=0. These are polynomial/value checks; no nonsingularity or cryptographic fixture claim is made.

The field dependency MixedGeneric was independently reviewed in the separate mixed-generic-independent-review packet. Its source hash and review manifest are bound in sources.json. This newly composed word packet still requires its own independent literal-source review. No scientific execution or performance measurement occurred; cost ratio is null.

Reproduction: copy this packet's source and checker into a new private directory, preserving dependency paths from check.py; invoke check.py WordGeneric UNIQUE_TAG, then Controls and Readback with fresh tags. Each invocation has a 120-second cap and writes its own source snapshot, native stdout/stderr and receipt. Source dependency hashes must match sources.json. Existing dependency oleans are reused; this is not a claim that all transitive dependencies were rebuilt here.